

Final Report SURGE-2025

Mentor: Prof. Sutanu Gayen
Mentee: Prithviraj Ghosh

July 2025

1 Introduction

The knapsack counting problem ($\#$ KNAP) is defined as follows: given a non-negative vector $a \in \mathbb{R}_+^n$ and a non-negative scalar $b \in \mathbb{R}_+$, count the number of binary vectors $x \in \{0, 1\}^n$ such that $\sum_i a_i x_i \leq b$. It is well-known that the $\#$ KNAP problem is $\#$ P-hard, and considerable attention has been devoted to approximating the size of $\text{KNAP}(a, b)$. Specifically, given an error parameter ε , the goal is to compute a value p such that $(1 - \varepsilon)|\text{KNAP}(a, b)| \leq p \leq (1 + \varepsilon)|\text{KNAP}(a, b)|$, in time polynomial in both n and $1/\varepsilon$. Such a value p is referred to as a *relative-error* ε -approximation.

2 Preliminaries

2.1 Read-Once Branching Program

An (S, T) -branching program M is a layered multi-graph with a layer for each $0 \leq i \leq T$, and at most S vertices (states) in each layer. The first layer contains a single vertex v_0 , and each vertex in the last layer is labeled with either 0 (rejecting) or 1 (accepting). For each $0 \leq i < T$, every vertex v in layer i has two outgoing edges labeled 0 and 1, each leading to vertices in layer $i + 1$.

Notations

Let M be an (S, T) -branching program and v a vertex in layer i of M .

1. For a string z , $M(v, z)$ denotes the state reached by starting from v and following edges labeled with z .

2. For $z \in \{0, 1\}^n$, define:

$$M(z) = \begin{cases} 1 & \text{if } M(v_0, z) \text{ is an accepting state} \\ 0 & \text{otherwise} \end{cases}$$

3. $A_M(v) = \{z : M(v, z) \text{ is accepting in } M\}$, and $P_M(v)$ is the probability that $M(v, z)$ is an accepting state when z is chosen uniformly at random.
4. $L(M, i)$ denotes the set of vertices in layer i of M .
5. For a set U , $x \sim U$ denotes a uniformly random element chosen from U .

2.2 Small Space Source

A width- w small-space source is described by a (w, n) -branching program D along with a probability distribution p_v on the outgoing edges associated with each vertex $v \in D$. Samples from the source are generated by performing a random walk on D , where the transitions follow the distributions p_v , and the output is the sequence of labels on the edges traversed.

3 Related Works

There are two papers which independently gave Fully Polynomial Time Approximation(FPTA) algorithms for the problem.

3.1 Polynomial-Time Approximation Schemes for Knapsack and Related Counting Problems using Branching Programs by Parikshit Gopalan, Adam Klivans, and Raghu Meka

In this paper an approach is discussed of creating an exact branching program from the weights in such a way that each state in layer j corresponds to a possible value of the partial sum $v_j = \sum_{i \leq j} a_i x_i$. Then the program is approximated to a small width where every element in each layer is chosen by constructing a series of branching programs $M^n = M, M^{n-1}, \dots, M^0$. We obtain M^i from M^{i+1} by rounding the states in $L(M^{i+1}, i+1)$. More precisely, we set $L(M^i, i+1) = \{v_1, \dots, v_\ell\} \subseteq L(M^{i+1}, i+1)$ where the v_j 's are defined as follows: let $v_1 = 0$. Given v_j , let

$$v_{j+1} = \min \left\{ v : v > v_j \text{ and } 0 < P_{M^{i+1}}(v) \leq \frac{P_{M^{i+1}}(v_j)}{1 + \varepsilon} \right\}$$

Since $P_{M^i}(v_1) \leq 1$ and $P_{M^i}(v_{\ell-1}) \geq 2^{-n}$, we have $\ell \leq O(n/\varepsilon)$. Next all the edges are redirected to v_j if it was initially directed to v where $v_j \leq v \leq v_{j+1}$.

It is then shown that each vertex $v_j \in L(M^i, i+1)$ can be computed in time $O(\log(n/\varepsilon) \log W)$. It is proved by first showing that we can inductively calculate the probabilities of each element to reach the accepting state and so we just have to search the child nodes in the next layer in $O(\log(n/\varepsilon))$ time. Then performing a binary search on the current layer with ordering as the probability (since it is monotonic) we can get each element of the current layer in $O(\log(n/\varepsilon) \log W)$. Hence We can construct M^0 from M in time $O\left(\frac{n^2 \log(W) \log(n/\varepsilon)}{\varepsilon}\right)$.

Then finally it is inductively shown that $P_M(s) \leq P_{M^0}(s) \leq (1 + \delta)P_M(s)$ where $\varepsilon = \Omega\left(\frac{\delta}{n}\right)$. This follows from the construction of the rounded branching program, where the probability of each retained element differs by at least a factor of $1 + \varepsilon$.

So the final time complexity of this algorithm is $O\left(\frac{n^3 \log(W) \log(n/\varepsilon)}{\varepsilon}\right)$

3.2 A Deterministic Polynomial-time Approximation Scheme for Counting Knapsack Solutions by Daniel Stefankovic, Santosh Vempala and Eric Vigoda

This paper defines a function $\tau : \{0, \dots, n\} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R} \cup \{\pm\infty\}$ where $\tau(i, a)$ is the smallest C such that there exist at least a solutions to the knapsack problem with weights $w_1, \dots, w_i$ and capacity C . Here the solution of the problem will be $\max\{a : \tau(n, a) \leq C\}$. Then a recursion is established for τ which is

$$\tau(i, a) = \min_{\alpha \in [0, 1]} \max \begin{cases} \tau(i-1, \alpha a) \\ \tau(i-1, (1-\alpha)a) + w_i \end{cases}$$

In order to approximate τ as we can not compute efficiently since the second argument ranges over all real numbers only the weights are considered that goes up a geometric series. For doing that Let

$$Q := 1 + \frac{\varepsilon}{n+1} \quad \text{and} \quad s := \lceil n \log_Q 2 \rceil.$$

The function

$$T : \{0, \dots, n\} \times \{0, \dots, s\} \rightarrow \mathbb{R}_{\geq 0} \cup \{\infty\}$$

is defined using the recurrence that the function τ satisfies. Namely, T is defined by the following recurrence:

$$T[i, j] = \min_{\alpha \in [0, 1]} \max \begin{cases} T[i-1, \lfloor j + \log_Q \alpha \rfloor] \\ T[i-1, \lfloor j + \log_Q 1 - \alpha \rfloor] + w_i \end{cases}$$

where the final solution will be $Q^{j'+1}$ where $j' = \max\{j : T[n, j] \leq C$
It specifically holds as the following inequality is established along with it which can be proved inductively.

$$\tau(i, Q^{j-i}) \leq T[i, j] \leq \tau(i, Q^j)$$

Also, using the fact that $T[i, j]$ is non decreasing in j , the first argument in the maximum (namely, $T[i - 1, \lfloor j + \log_Q \alpha \rfloor]$) is non-decreasing in α . Similarly, the second argument in the maximum is a non-increasing function in α . Hence, the minimum of the maximum of the two arguments occurs either at the boundary (that is, for $\alpha \in \{0, 1\}$) or for $\alpha \in (0, 1)$ where the derivative changes from negative to positive. This makes us calculate the elements of the table is $\log s$ time making the total time complexity of the algorithm $O(ns \log s)$ which is same as $O\left(\frac{n^3 \log(n/\varepsilon)}{\varepsilon}\right)$

4 Experimentation

We implemented the code for both of the algorithms where we observed that the branching program approach is giving the output in better time for relatively small values of W where the main reason is the fact that the number of elements in each layer is much less than $\frac{n^2}{e}$ while in the other the number of elements in the table is always $\frac{n^2}{e}$

The code can be seen here : [\[9\]](#)

Input Parameters

- Number of weights: 25
- Weights (w_i):
854327, 1053612, 1098036, 1542999, 1014851, 1831154, 642235, 1234009, 1421891, 683390,
1658233, 1447844, 1086915, 1164021, 1641211, 752760, 687661, 1131533, 1762525, 1654152,
655566, 1681616, 717056, 974577, 986033
- Capacity (C): 15,862,646
- Approximation parameter (ε): 0.01

Solutions and Timing

Method	Approximate Count	Time Taken
Branching Program	21,713,797.0	3.3 s
Dynamic Programming	21,693,448.864	22 s
Exact Solution	21,680,945	25 s

Table 1: Comparison of solutions and their computation time

5 Improved Branching Program for Counting Knapsack

5.1 Proposal

Quoting from the ROBP paper "We can construct M^0 from M in time $O\left(n^2 \log(W) \log\left(\frac{n}{\varepsilon}\right) / \varepsilon\right)$." I propose using the dp algorithm learnt from the second paper we can construct M^0 from M in time $O\left(n^3 \log\left(\frac{n}{\varepsilon}\right) / \varepsilon\right)$

5.2 Proof

5.2.1 Idea

According to the claim 3.2 of the ROBP paper "Each vertex $v_j \in L(M^i, i+1)$ can be computed in time $O(\log(\frac{n}{\varepsilon}) \log W)$." I try to reduce it to $O(\log(\frac{n}{\varepsilon}))$ using a precomputed dp table as used in the other paper. We will use a similar definition of τ and T appropriately for doing the approximation.

5.2.2 Defining τ

Consider we have created M^{i+1} and creating M^i and like the assumption of the claim "We have the vertices v_j of $L(M^i, i+1)$ stored in a binary tree, and also know their acceptance probabilities $P_{M^i}(\cdot)$." So by the method of rounding used to create M^i 0 must be the first vertex in layer i and the probability of which can be calculated in $O(\log(\frac{n}{\varepsilon}))$ as discussed in the paper. Now for the upcoming vertices in the layer we will use the following definition of τ :

$$\tau : \{0, \dots, n\} \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R} \cup \{\pm\infty\}$$

where $\tau(i, a)$ is defined as is the smallest C such that there exist at least a solutions to the knapsack problem with weights $w_{i+1}, \dots, w_n$ and capacity C which is the same definition as the paper just with different weights as the orientation of the weights did not matter in the analysis later

Claim: Assume we have calculated v_j in the layer i along with its probability to reach an accepting state p and want to find v_{j+1} then the following equation holds exactly

$$v_{j+1} = C - \tau\left(i, \frac{2^{n-i}p}{1+\varepsilon}\right) + 1$$

Proof: By the definition of τ the expression $C - \tau\left(i, \frac{2^{n-i}p}{1+\varepsilon}\right)$ is the maximum v such that the probability of v to reach the accepting state is at least $\frac{p}{1+\varepsilon}$. So by adding 1 we get exactly v_{j+1} by the algorithm of rounding discussed in the paper.

5.2.3 Approximating τ

Consider we have the T table as mentioned in the paper and creating that took time $O(n^3 \log(\frac{n}{\varepsilon})/\varepsilon)$ then we can take the value $T\left[i, \left\lfloor \log_Q\left(\frac{2^{n-i}p}{1+\varepsilon}\right) \right\rfloor\right]$ as an appropriate approximation of $\tau\left(i, \frac{2^{n-i}p}{1+\varepsilon}\right)$ as by the lemma 2.2 of the dp paper

$$T\left[i, \left\lfloor \log_Q\left(\frac{2^{n-i}p}{1+\varepsilon}\right) \right\rfloor\right] \leq \tau\left(i, Q^{\left\lfloor \log_Q\left(\frac{2^{n-i}p}{1+\varepsilon}\right) \right\rfloor}\right) \leq \tau\left(i, \frac{2^{n-\ell}p}{1+\varepsilon}\right)$$

and

$$\begin{aligned} T\left[i, \left\lfloor \log_Q\left(\frac{2^{n-i}p}{1+\varepsilon}\right) \right\rfloor\right] &\geq \tau\left(i, Q^{\left\lfloor \log_Q\left(\frac{2^{n-i}p}{1+\varepsilon}\right) \right\rfloor - n + i}\right) \\ &\geq \tau\left(i, Q^{\log_Q\left(\frac{2^{n-i}p}{1+\varepsilon}\right) - n + i - 1}\right) \\ &= \tau\left(i, \frac{2^{n-\ell}p}{Q^{n-i+1}(1+\varepsilon)}\right) \\ &\geq \tau\left(i, \frac{2^{n-\ell}p}{(1+\varepsilon)^2}\right) \geq \tau\left(i, \frac{2^{n-\ell}p}{1+2\varepsilon}\right) \end{aligned}$$

So

$$\tau\left(i, \frac{2^{n-\ell}p}{1+2\varepsilon}\right) \leq T\left[i, \left\lfloor \log_Q\left(\frac{2^{n-i}p}{1+\varepsilon}\right) \right\rfloor\right] \leq \tau\left(i, \frac{2^{n-\ell}p}{1+\varepsilon}\right)$$

Hence by putting it in the claim in above we get the approximate v_{j+1} calling it $(v_{j+1})_{\text{approx}}$ lies between the two approximations always

$$(v_{j+1})_{\varepsilon} \leq (v_{j+1})_{\text{approx}} \leq (v_{j+1})_{2\varepsilon}$$

where $(v_{j+1})_\varepsilon$ is the next vertex with approximation factor ε and $(v_{j+1})_{2\varepsilon}$ is the next vertex with approximation factor 2ε .

Therefore, by monotonicity of the probability

$$\frac{p}{1+\varepsilon} \geq p_{v_{j+1}} \geq \frac{p}{1+2\varepsilon}$$

Hence the approximation factor is just a small constant multiple of ε which makes it is similar to the given approximation in the paper with time complexity $O\left(n^3 \log\left(\frac{n}{\varepsilon}\right) / \varepsilon + n^2 \log\left(\frac{n}{\varepsilon}\right) / \varepsilon\right) = O\left(n^3 \log\left(\frac{n}{\varepsilon}\right) / \varepsilon\right)$

6 Dynamic Programming for Counting Knapsack with fixed Hamming Weight

We have a Small Space Source which is the uniform distribution over the width $n+1$ where any node s representing the hamming weight in layer i is connected with node s and node $s+1$ in layer $i+1$ each representing not accepting and accepting w_{i+1} respectively. Let the fixed Hamming weight be k .

We define $\tau(i, a, s)$ as the minimum capacity with the accepting probability over all possible binary strings is at least a and present state in the small space source is s at layer i for weights $w_1, w_2, \dots, w_i$.

Claim

$$\tau(i, a, s) = \min_{\substack{\alpha, \beta \in [0,1] \\ \alpha p_1 + \beta p_2 = a}} \max \left\{ \begin{array}{l} \tau(i-1, \alpha, s) \\ \tau(i-1, \beta, s-1) + w_i \end{array} \right.$$

where p_1 & p_2 are the probabilities associated with the edge from s and $s-1$ in layer $i-1$ of the small space source respectively.

Proof

Fix any $\alpha \in [0, 1]$ and automatically a β is fixed.

Call $B = \max\{\tau(i-1, \alpha, s), \tau(i-1, \beta, s-1) + w_i\}$. There is at least α probability with weights $w_1, w_2, \dots, w_{i-1}$ with present state s and capacity B . Also there is at least β probability with weights $w_1, w_2, \dots, w_{i-1}$ with present state $s-1$ and capacity $B - w_i$. Since $\alpha p_1 + \beta p_2 = a$ there is at least a probability with weights $w_1, w_2, \dots, w_i$ with present state s and capacity B as all solutions are disjoint. So,

$$\tau(i, a, s) \leq \min_{\substack{\alpha, \beta \in [0,1] \\ \alpha p_1 + \beta p_2 = a}} \max \left\{ \begin{array}{l} \tau(i-1, \alpha, s) \\ \tau(i-1, \beta, s-1) + w_i \end{array} \right.$$

Calling $\tau(i, a, s) = C$. Let α be such that αp_1 be the fraction of a which does not include w_i and β accordingly. So, $\tau(i-1, \alpha, s) \leq C$ and $\tau(i-1, \beta, s-1) \leq C - w_i$. Hence $\max\{\tau(i-1, \alpha, s), \tau(i-1, \beta, s-1) + w_i\} \leq C$

$$\tau(i, a, s) \geq \min_{\substack{\alpha, \beta \in [0,1] \\ \alpha p_1 + \beta p_2 = a}} \max \left\{ \begin{array}{l} \tau(i-1, \alpha, s) \\ \tau(i-1, \beta, s-1) + w_i \end{array} \right.$$

Giving the equality.

The solution probability $a^* = \max\{a : \tau(n, a, k) \leq C\}$

As we have got

$$\tau(i, a, s) = \min_{\substack{\alpha, \beta \in [0,1] \\ \alpha p_1 + \beta p_2 = a}} \max \left\{ \begin{array}{l} \tau(i-1, \alpha, s) \\ \tau(i-1, \beta, s-1) + w_i \end{array} \right.$$

Using x, y such that $\alpha = xa$ and $\beta = ya$ we can rewrite the expression as

$$\tau(i, a, s) = \min_{\substack{x, y \in [0,1] \\ xp_1 + yp_2 = 1}} \max \left\{ \begin{array}{l} \tau(i-1, xa, s) \\ \tau(i-1, ya, s-1) + w_i \end{array} \right.$$

In order to approximate τ we use T which has 3 parameters i, j, s where all of them are natural numbers with $i \in [0, n]$, $j \in [0, k]$ where $k = \lceil n \log_{1+\delta} n \rceil$ and $s \in [0, n]$ where

$$T[i, j, s] = \min_{\substack{x, y \in [0,1] \\ xp_1 + yp_2 = 1}} \max \left\{ \begin{array}{l} T[i-1, \lfloor j + \log_{1+\delta} x \rfloor, s] \\ T[i-1, \lfloor j + \log_{1+\delta} y \rfloor, s-1] + w_i \end{array} \right.$$

Claim

Take $r = \frac{1}{n^n}$ which is the minimum possible probability of a solution in the hamming weight small space source. Let $i \geq 1$. Assume that for all $j \in \{0, \dots, k\}$ and $s \in \{0, \dots, n\}$ we have that $T[i-1, j]$ satisfies the below condition. Then for all $j \in \{0, \dots, k\}$ and $s \in \{0, \dots, n\}$ we have that $T[i, j]$ computed satisfies:

$$\tau(i, r(1+\delta)^{j-i}, s) \leq T[i, j, s] \leq \tau(i, r(1+\delta)^j, s)$$

Proof

As we have assumed

$$\begin{aligned} T[i-1, \lfloor j + \log_{1+\delta} x \rfloor, s] &\leq \tau(i-1, r(1+\delta)^{\lfloor j + \log_{1+\delta} x \rfloor}, s) \\ &\leq \tau(i-1, r(1+\delta)^{j+\log_{1+\delta} x}, s) \\ &= \tau(i-1, xr(1+\delta)^j, s) \end{aligned}$$

and

$$\begin{aligned} T[i-1, \lfloor j + \log_{1+\delta} y \rfloor, s-1] &\leq \tau(i-1, r(1+\delta)^{\lfloor j + \log_{1+\delta} y \rfloor}, s-1) \\ &\leq \tau(i-1, r(1+\delta)^{j+\log_{1+\delta} y}, s-1) \\ &= \tau(i-1, yr(1+\delta)^j, s-1) \end{aligned}$$

Hence

$$\begin{aligned} T[i, j, s] &= \min_{\substack{x, y \in [0, 1] \\ xp_1 + yp_2 = 1}} \max \left\{ \begin{array}{l} T[i-1, \lfloor j + \log_{1+\delta} x \rfloor, s] \\ T[i-1, \lfloor j + \log_{1+\delta} y \rfloor, s-1] + w_i \end{array} \right\} \\ &\leq \min_{\substack{x, y \in [0, 1] \\ xp_1 + yp_2 = 1}} \max \left\{ \begin{array}{l} \tau(i-1, xr(1+\delta)^j, s) \\ \tau(i-1, yr(1+\delta)^j, s-1) + w_i \end{array} \right\} = \tau(i, r(1+\delta)^j, s) \end{aligned}$$

Also

$$\begin{aligned} T[i-1, \lfloor j + \log_{1+\delta} x \rfloor, s] &\geq \tau(i-1, r(1+\delta)^{\lfloor j + \log_{1+\delta} x \rfloor - i + 1}, s) \\ &\geq \tau(i-1, r(1+\delta)^{j+\log_{1+\delta} x - i}, s) \\ &= \tau(i-1, xr(1+\delta)^{j-i}, s) \end{aligned}$$

and

$$\begin{aligned} T[i-1, \lfloor j + \log_{1+\delta} y \rfloor, s-1] &\geq \tau(i-1, r(1+\delta)^{\lfloor j + \log_{1+\delta} y \rfloor - i + 1}, s-1) \\ &\geq \tau(i-1, r(1+\delta)^{j+\log_{1+\delta} y - i}, s-1) \\ &= \tau(i-1, yr(1+\delta)^{j-i}, s-1) \end{aligned}$$

Hence

$$\begin{aligned} T[i, j, s] &= \min_{\substack{x, y \in [0, 1] \\ xp_1 + yp_2 = 1}} \max \left\{ \begin{array}{l} T[i-1, \lfloor j + \log_{1+\delta} x \rfloor, s] \\ T[i-1, \lfloor j + \log_{1+\delta} y \rfloor, s-1] + w_i \end{array} \right\} \\ &\geq \min_{\substack{x, y \in [0, 1] \\ xp_1 + yp_2 = 1}} \max \left\{ \begin{array}{l} \tau(i-1, xr(1+\delta)^{j-i}, s) \\ \tau(i-1, yr(1+\delta)^{j-i}, s-1) + w_i \end{array} \right\} = \tau(i, r(1+\delta)^{j-i}, s) \end{aligned}$$

And hence our claim is proved.

Solution

Call $j' = \max\{j : T[n, j, k] \leq C\}$ then $r(1+\delta)^{j'+1}$ can be an appropriate approximation of the problem. As $C < T[n, j'+1, k] \leq \tau(n, r(1+\delta)^{j'+1}, k)$ which means the required probability is at most $r(1+\delta)^{j'+1}$. And also

$\tau(n, r(1+\delta)^{j'-n}, k) \leq T[i, j', k] \leq C$ which implies the required probability is at least $r(1+\delta)^{j'-n}$

Let p' be the approximated probability and p be the exact probability. Then

$$\frac{p'}{p} \leq \frac{r(1+\delta)^{j'+1}}{r(1+\delta)^{j'-n}} = (1+\delta)^{n+1}$$

Taking $\varepsilon = O(\frac{\delta}{n})$ we get a ε approximate solution of the problem

Running time Complexity

The size of the dp table is $O(n^2k)$ where k is $O(\frac{n^2 \log n}{\varepsilon})$ and since when we fix one of x or y then one among $\lfloor j + \log_{1+\delta} x \rfloor$ or $\lfloor j + \log_{1+\delta} y \rfloor$ increases and the other decreases or vice versa. So we can use the technique mentioned in the paper to optimize the calculation of each element of the table from $O(k)$ to $O(\log k)$ and hence total complexity becomes $O(n^2k \log k)$

7 Conclusion

The approach of the dynamic programming fixed Hamming weight can be used for improving the branching program approach which gives efficient random samples and hence has more practical use. Also, there is a scope for more generalisation of the problem where there will be more than 2 parent nodes for a node in the small space source which can be explored when extended further.